# Searching for Exoplanets using a Support Vector Machine

Gene Foxwell

Wolf University / Udacity School of Artificial Intelligence

Applied Machine Learning

## Overview

This project looks at building a Machine Learning Model for predicting if a Star has an Exoplanet based on flux observations from the Kepler Telescope. We will provide an overview of the problem and the dataset. Then we will go over other approaches to this problem. Next, we will build a preprocessing pipeline aimed at performing dimensionality reduction. Finally, we will cover the training of the Support Vector Machine Model and discuss the results.

## Dataset

We are using the version of this dataset hosted by Kaggle. The dataset is provided in two parts: A training dataset, and A test dataset. Both datasets are extremely unbalanced – the training dataset has 5087 rows of which only 37 represents planets. The test dataset has 571 rows, of which only 5 are planets! The challenges this present will be discussed later in this paper.

Each row of the dataset has 3198 columns. The first column is the label. 1 -> Low probability of a planet around the candidate star, 2 -> High probability of planet around the candidate star. The remaining 3197 columns are the flux observations as recorded from the Kepler Telescope.

Flux, in the context of astronomy, is defined as the amount of energy received from a celestial object per unit area, per unit time. It is effectively a way of measuring how much energy is received from the star.

The idea behind planet detection in this context is that when an exoplanet passes between the Star and Earth, the flux we detect from that star will decrease as some of that flux has been blocked off by the exoplanet.

As we can see from figure 1, the flux data collected is extremely noisy. Much of our work in the next section will focus on cleaning up this noise to enable the Machine Learning Model to work effectively.

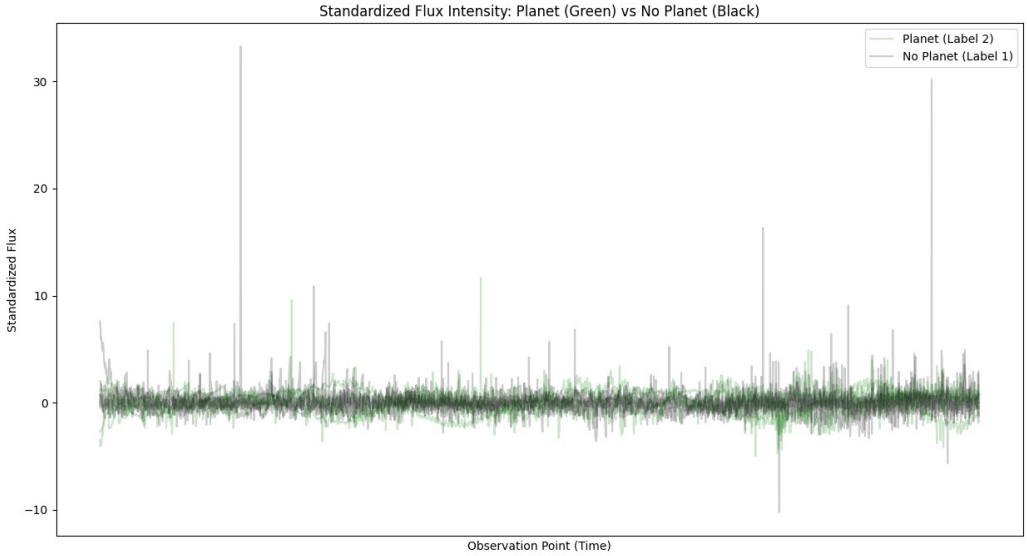


*Figure 1 Same of Planet vs. No Planet Observations*

## Other Approaches to the Problem

While researching this dataset, we came across other approaches to the Kepler dataset. (Airlangga, G. 2024) approach comparing the results of a variety of ML Models. It did not appear from either paper that they attempted dimensionality reduction akin to our approach here – although the precise details of the preprocessing done are shallow in both papers. Both mention

the use of a Fast Fourier Transform (double check), an approach which we also take here, before preforming our dimensionality reducing operation.

## Modeling Approach

For our approach we focused on creating a reduced dimensional set of features for input into a Support Vector Machine (Cortes 1995). Our general idea was to find a way to encode the fact that planets induce a reduction in flux directly into our features. To this end, our prepossessing and feature engineering pipeline consisted of the following steps:

1. *Fast Fourier Transform (FFT):* We used scipy's rfft function to compute a 1-D Fast Fourier Transform (Cooley 1965) for each row. This effectively converted our domain to the frequency domain. The idea here is that a consistent "dip" in flux measurements would show up as change in the frequency response.
2. *Binned Z-Scores:* This was the primary means via which we reduced the dimensionality of the data. For each row we calculated the mean of the row. Then we divided the row into bins of 16 observations each. We returned a Z-Score for each bin. The Z-Score is the number of standard deviations a value is above or below the mean. This resulted in dimensionality reduction of 3197 features down to 100 features.
3. *Standardization:* This step transformed the data, so it has mean 0 and standard deviation 1. This was used to keep the data roughly in the same scale to avoid larger values biasing the model.

Note: Scaling and Standardization combined have been shown to improve classification accuracy in machine learning models. Cao 2016 provides an example of this as applied to bio-medical data.

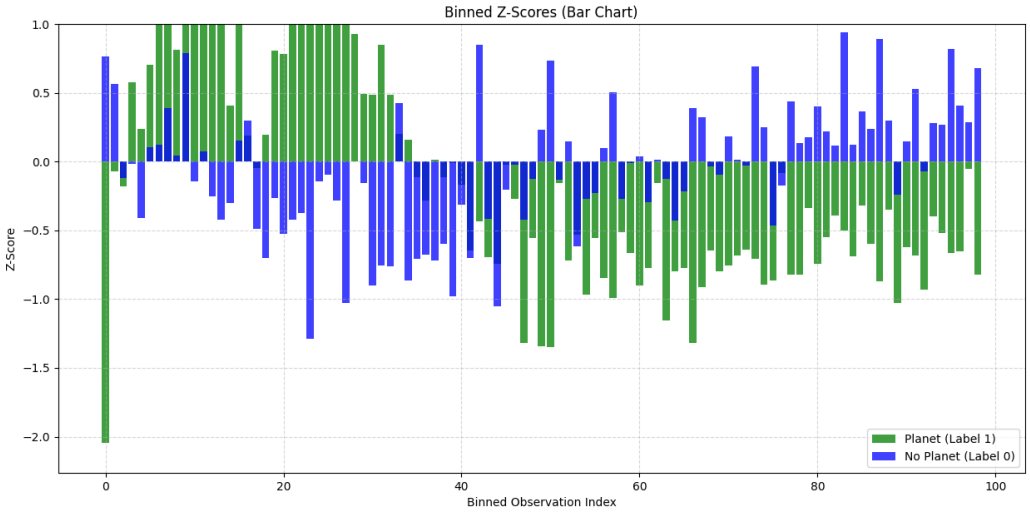


*Figure 2 Binned Z-Score Visualization*

Figure 2 provides a visualization of the features after prepossessing, showing results for a Star both with and without an Exoplanet. We can see evidence of a "dip" two standard deviations below the mean right at the start of the observation. (Shown in green)

For our initial model, inspired by (Chawla 2002), we over-sampled the positive examples in the dataset. With oversampling, our results before fine-tuning can been seen in figure 3:

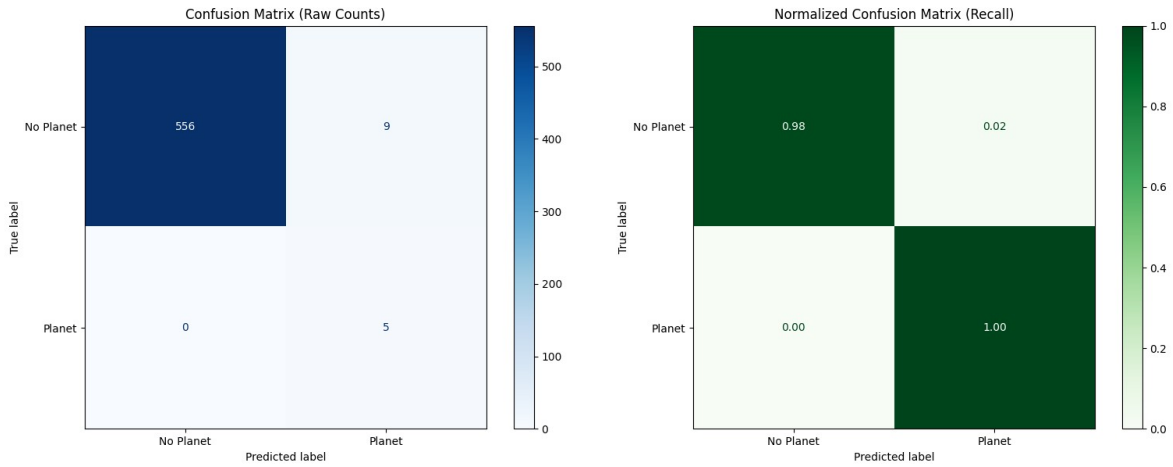


*Figure 3: Initial Confusion Matrix*

After testing the initial model with the transformed features, we fine-tuned the Support Vector Machine using Grid Search CV. We used the following parameters for our search:

| **Parameter** | **Values** |
|---|---|
| **C** | 1.0, 0.1, 0.05, 0.005, 0.001, 0.0001 |
| **gamma** | 1.0, 0.1, 0.01, 0.001, 0.0001, ‘scale’ |

To account for the extreme class imbalance seen in this dataset, we trained GridSearchCV[1] using a StratifiedKFold[2] strategy, which ensured each fold maintains the same class distribution as the original dataset. Otherwise, it was possible that some folds might not have any positive examples in them.

To keep consistent with how our original model was trained, we instructed GridSearchCV to use balanced class weights for training.

[1] GridSearchCV — scikit-learn 1.8.0 documentation
[2] StratifiedKFold — scikit-learn 1.8.0 documentation

We used a RBF kernel and precision score as the evaluation metric. The use of RBF is recommended as a starting point in Hsu 2016 due to its ability to handle non-linear relationship between features and labels.

After Grid Search was completed, the best-found parameters were C: 1, gamma: 0.01. These parameters were used to generate the results discussed in the next section.

## Results

As seen in the final confusion matrix (figure 4) the system correctly identified all but two of the planets in the "No Planet" category of the test data. For stars with a high probability of having an Exoplanet, the system correctly identified 4 out of 5 data points.

Given the extreme imbalance of the dataset, the standard metrics, F1 Score, Accuracy, Precision, all appeared somewhat unreliable. For example, it was possible to obtain a much higher F1 score by simply declaring all stars as having no planets. The same situation for accuracy and recall.

The table below summarizes the metrics collected from the final model:

| **Metric** | **Score** |
|---|---|
| **F1 Score** | 0.72 |
| **Accuracy** | 0.994 |
| **Precision** | 0.67 |
| **Recall** | 0.8 |

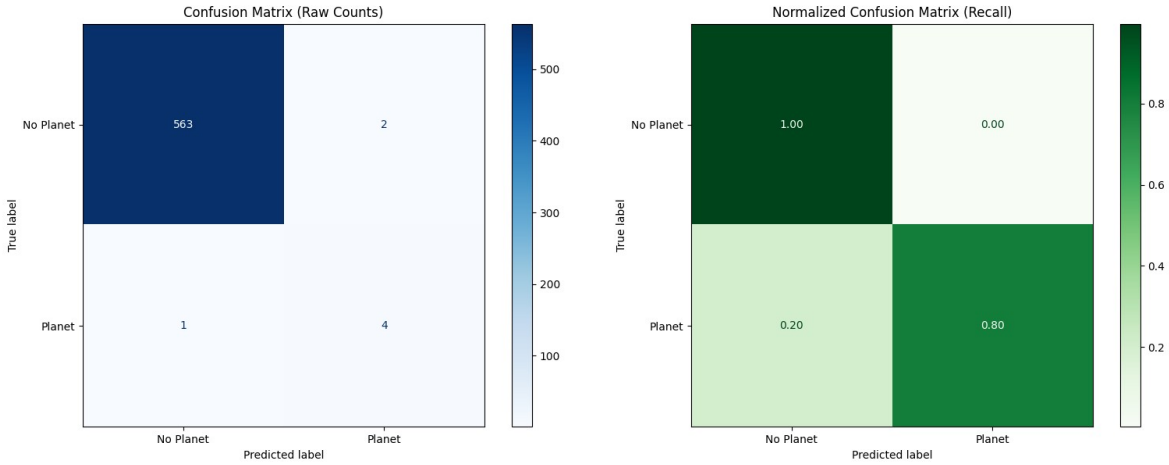


*Figure 4 Final Confusion Matrix*

## Non-Technical Interpretation

We built a model capable of predicting if a star has a high probability of having an exoplanet based on observations from the Kepler telescope. Due to there being so few known exoplanets at the time our data was sourced, we had very few examples to work with. As a result, we had trouble providing a reliable measurement of the models effectiveness. Despite this, our model was shown to be able to recognize 80% of the exoplanets in our training data.

As far as determine stars that did not have an exoplanet, our model achieved a near perfect result – mistaking only two entries in the dataset for having an exoplanet when they did not.

## Limitations and Biases

As noted elsewhere in this paper, the dataset we worked with was extremely unbalanced. This not only presented challenges in training out model, but in measuring its performance as

well. With so few positive observations in the training and test set, it is difficult to be confident in the ability of the model to generalize to future measurements. This model should not be used as the final determination for declaring whether a star has an Exoplanet or not. Instead, it would be better used as indicator for further investigation.

Information on exoplanets collected by the Kepler telescope is biased by what we can observe – we can only detect exoplanets on stars who geometry would place the exoplanet between Kepler and the star. Planets that do not meet this criterion cannot be detected. Future work should look for ways to validate this methodology using other methods of detection or other telescope configurations.

## References

Airlangga, G. (2024). Exoplanet Classification Through Machine Learning: A Comparative Analysis of Algorithms Using Kepler Data. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, *4*(3), 753-763. https://doi.org/10.57152/malcom.v4i3.1303

Cao XH, Stojkovic I, Obradovic Z. A robust data scaling algorithm to improve classification accuracies in biomedical data. BMC Bioinformatics. 2016 Sep 9;17(1):359. doi: 10.1186/s12859-016-1236-x. PMID: 27612635; PMCID: PMC5016890.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation, 19*(90), 297–301. https://web.stanford.edu/class/cme324/classics/cooley-tukey.pdf

Cortes, C., & Vapnik, V.N. (1995). Support-Vector Networks. *Machine Learning, 20*, 273-297. https://www.semanticscholar.org/paper/Support-Vector-Networks-Cortes-Vapnik/52b7bf3ba59b31f362aa07f957f1543a29a4279e

Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2016). *A practical guide to support vector classification*. Department of Computer Science, National Taiwan University. https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf